

0.1. Important exercises

Make sure you have already finished:

- `session-5/tests/error-handling.rs`

1. Threads	2
1.1. Plain Threads	3
1.2. Receiving data from threads	5
1.3. Handling thread panics	6
1.4. Sending references to threads	7
1.5. Borrowing and threads (continued)	8
1.6. Scoped threads	9
1.7. Multiple scoped threads	10
1.8. Moving to threads	11
1.9. Review: what is 'static'?	12
2. Channels	13
3. Channels	14
4. Send and Sync	19
5. Shared state	28
6. Exercises	35

1.1. Plain Threads

```
1  use std::thread;
2  use std::time::Duration;
3
4  fn main() {
5      thread::spawn(|| {
6          for i in 0..10 {
7              println!("Count in thread: {i}!");
8              thread::sleep(Duration::from_millis(5));
9          }
10     });
11
12     for i in 0..5 {
13         println!("Main thread: {i}");
14         thread::sleep(Duration::from_millis(5));
15     }
16 }
```

(playground link)

What happens to the spawned thread?

1.1. Plain Threads

```
1  use std::thread;
2  use std::time::Duration;
3
4  fn main() {
5      thread::spawn(|| {
6          for i in 0..10 {
7              println!("Count in thread: {i}!");
8              thread::sleep(Duration::from_millis(5));
9          }
10     });
11
12     for i in 0..5 {
13         println!("Main thread: {i}");
14         thread::sleep(Duration::from_millis(5));
15     }
16 }
```

(playground link)

What happens to the spawned thread?

Terminated by operating system when main thread exits. No destructors called.

1.1. Plain Threads

```
1 thread::spawn(|| {  
2     for i in 0..10 {  
3     });  
4  
5 for i in 0..5 {  
6     println!("Main thread: {i}");  
7     thread::sleep(Duration::from_millis(5));  
8 }
```

([playground link](#))

How can we ensure the spawned thread finishes?

1.1. Plain Threads

```
1 thread::spawn(|| {  
2     for i in 0..10 {  
3     });  
4  
5 for i in 0..5 {  
6     println!("Main thread: {i}");  
7     thread::sleep(Duration::from_millis(5));  
8 }
```

[\(playground link\)](#)

How can we ensure the spawned thread finishes?

Use `JoinHandle` returned by `thread::spawn` and call `join()` on it.

```
1 fn main() {  
2     let handle = thread::spawn(|| {  
3         // thread code  
4     });  
5     // main thread code  
6     handle.join();  
7 }
```

[\(playground link\)](#)

1.2. Receiving data from threads

How can we get a return value from a thread?

1.2. Receiving data from threads

How can we get a return value from a thread?

The `JoinHandle` returns a `Result` with the thread's return value.

```
1 fn main() {  
2     let handle = thread::spawn(|| {  
3         42  
4     });  
5  
6     let result = handle.join().unwrap();  
7     println!("The answer is: {}", result);  
8 }
```

(playground link)

What happens to the main thread if a spawned thread panics?

1.2. Receiving data from threads

How can we get a return value from a thread?

The `JoinHandle` returns a `Result` with the thread's return value.

```
1 fn main() {  
2     let handle = thread::spawn(|| {  
3         42  
4     });  
5  
6     let result = handle.join().unwrap();  
7     println!("The answer is: {}", result);  
8 }
```

(playground link)

What happens to the main thread if a spawned thread panics?

Nothing, the main thread continues executing.

1.3. Handling thread panics

If we don't join, we never see the panic.

1.3. Handling thread panics

If we don't join, we never see the panic.

If we don't handle the `Result` from `join()`, the panic is also ignored.

1.3. Handling thread panics

If we don't join, we never see the panic.

If we don't handle the `Result` from `join()`, the panic is also ignored.

We need to check the `Result` from `join()` to see if the thread panicked:

```
1  fn main() {  
2      let handle = thread::spawn(|| {  
3          panic!("Thread panicked!");  
4      });  
5      let result = handle.join();  
6      match result {  
7          Ok(_) => println!("Thread completed successfully."),  
8          Err(e) => println!("Thread panicked: {:?}", e),  
9      }  
10 }
```

[\(playground link\)](#)

Warning

You are responsible for handling panics in spawned threads! Rust will not do this for you.

Exercise:

- `examples/join-handle.rs`

1.4. Sending references to threads

Not all references can be sent to threads.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         let sum: i32 = data.iter().sum();  
5         println!("Sum: {sum}");  
6     });  
7     data.push(4); // modify while thread may be reading  
8     handle.join().unwrap();  
9 }
```

(playground link)

What happens if a thread borrows data from the main thread?

1.4. Sending references to threads

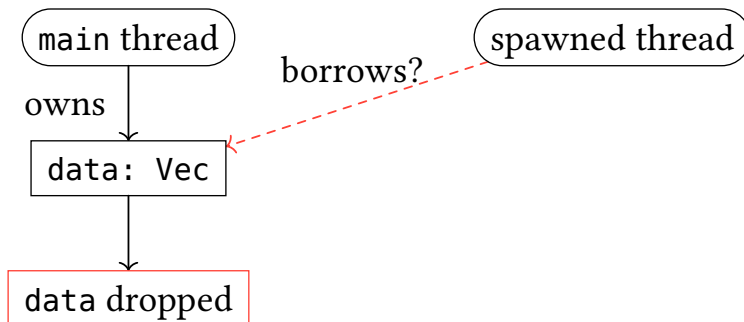
Not all references can be sent to threads.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         let sum: i32 = data.iter().sum();  
5         println!("Sum: {sum}");  
6     });  
7     data.push(4); // modify while thread may be reading  
8     handle.join().unwrap();  
9 }
```

[\(playground link\)](#)

What happens if a thread borrows data from the main thread?

Compilation error: closure may outlive the current function, but it borrows data.



1.4. Sending references to threads

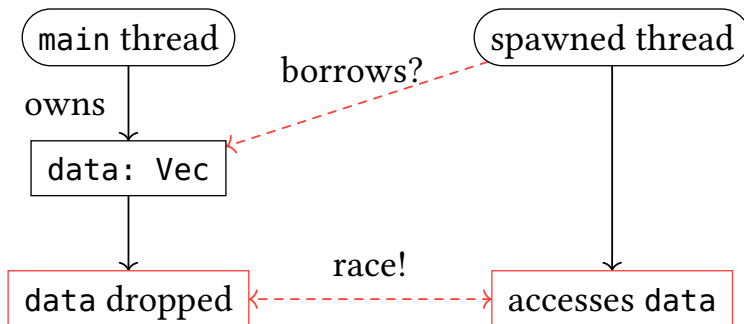
Not all references can be sent to threads.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         let sum: i32 = data.iter().sum();  
5         println!("Sum: {sum}");  
6     });  
7     data.push(4); // modify while thread may be reading  
8     handle.join().unwrap();  
9 }
```

([playground link](#))

What happens if a thread borrows data from the main thread?

Compilation error: closure may outlive the current function, but it borrows data.



1.5. Borrowing and threads (continued)

First attempt: ensure the thread finishes before main exits.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         let sum: i32 = data.iter().sum();  
5         println!("Sum: {sum}");  
6     });  
7     handle.join().unwrap();  
8     println!("Original: {:?}", data); // try to use after thread  
9 }
```

(playground link)

Does adding `join()` fix the borrow error?

1.5. Borrowing and threads (continued)

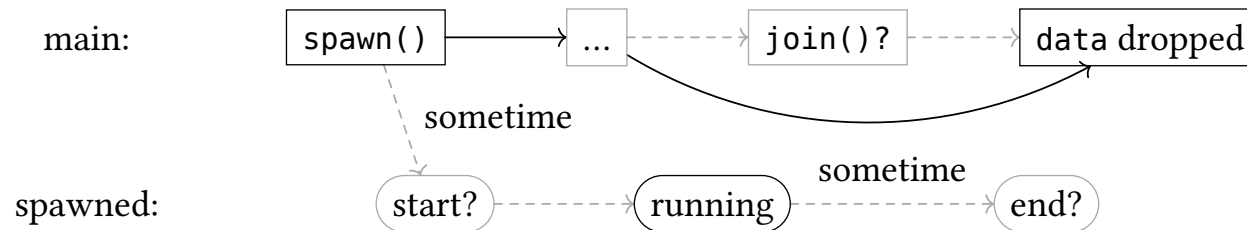
First attempt: ensure the thread finishes before main exits.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         let sum: i32 = data.iter().sum();  
5         println!("Sum: {sum}");  
6     });  
7     handle.join().unwrap();  
8     println!("Original: {:?}", data); // try to use after thread  
9 }
```

(playground link)

Does adding `join()` fix the borrow error?

No. Rust cannot verify at compile time that `join` is called before `data` goes out of scope.



1.5. Borrowing and threads (continued)

First attempt: ensure the thread finishes before main exits.

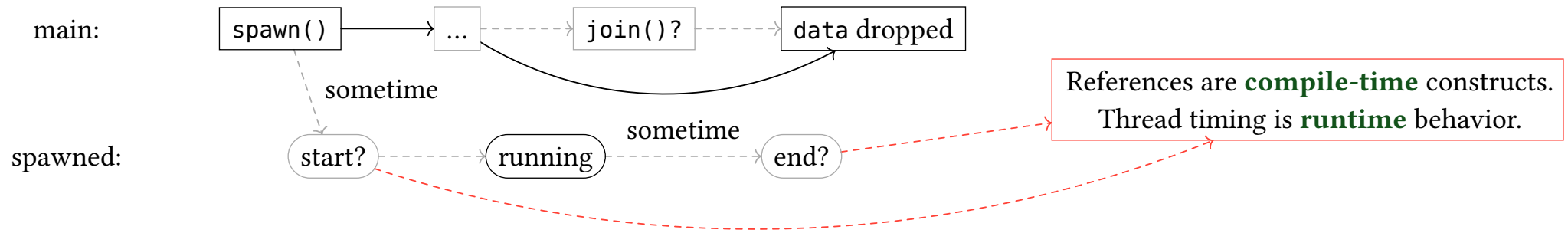
```

1 fn main() {
2     let data = vec![1, 2, 3];
3     let handle = thread::spawn(|| {
4         let sum: i32 = data.iter().sum();
5         println!("Sum: {sum}");
6     });
7     handle.join().unwrap();
8     println!("Original: {:?}", data); // try to use after thread
9 }
```

(playground link)

Does adding `join()` fix the borrow error?

No. Rust cannot verify at compile time that `join` is called before `data` goes out of scope.



1.6. Scoped threads

The problem: `thread::spawn` requires 'static because the thread may outlive the caller.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         println!("Data: {:?}", data); // error: `data` does not live long enough  
5     });  
6     handle.join().unwrap();  
7 }
```

([playground link](#))

1.6. Scoped threads

The problem: `thread::spawn` requires `'static` because the thread may outlive the caller.

```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     let handle = thread::spawn(|| {  
4         println!("Data: {:?}", data); // error: `data` does not live long enough  
5     });  
6     handle.join().unwrap();  
7 }
```

[\(playground link\)](#)

Solution: `thread::scope` guarantees all spawned threads complete before the scope ends.

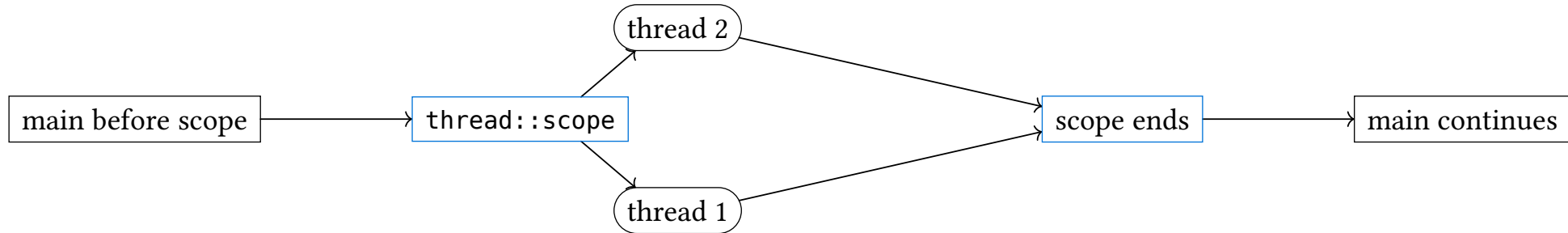
```
1 fn main() {  
2     let data = vec![1, 2, 3];  
3     thread::scope(|s| {  
4         s.spawn(|| println!("Data: {:?}", data)); // borrowing works!  
5     }); // all threads joined here automatically  
6     println!("Back in main: {:?}", data);  
7 }
```

[\(playground link\)](#)

1.7. Multiple scoped threads

```
1  fn main() {  
2      let mut data = vec![1, 2, 3];  
3  
4      thread::scope(|s| {  
5          s.spawn(|| println!("Thread 1: {:?}", data)); // shared borrow  
6          s.spawn(|| println!("Thread 2: {:?}", data)); // shared borrow  
7      });  
8  
9      data.push(4); // mutable access after scope ends  
10 }
```

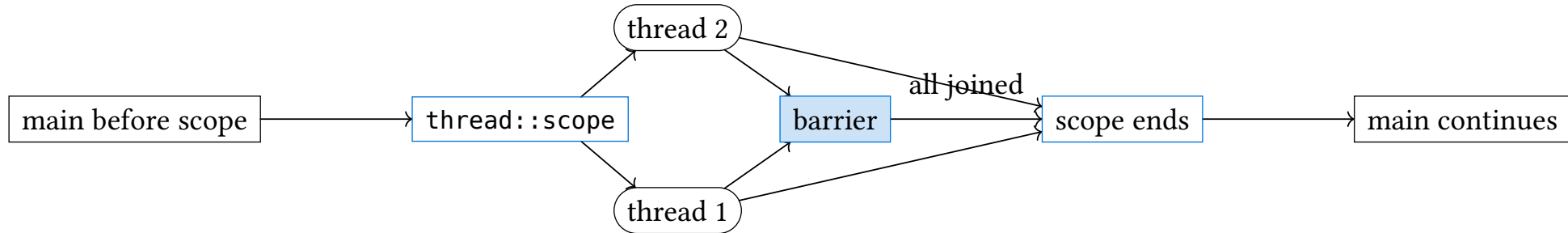
(playground link)



1.7. Multiple scoped threads

```
1  fn main() {  
2      let mut data = vec![1, 2, 3];  
3  
4      thread::scope(|s| {  
5          s.spawn(|| println!("Thread 1: {:?}", data)); // shared borrow  
6          s.spawn(|| println!("Thread 2: {:?}", data)); // shared borrow  
7      });  
8  
9      data.push(4); // mutable access after scope ends  
10 }
```

(playground link)



1.8. Moving to threads

In practice, scoped threads are not used very often.

The most common use case is to move ownership of data to the thread using the `move` keyword to transfer ownership at spawn time

```
1 fn main() {  
2     let data = String::from("hello");  
3     let handle = thread::spawn(move || {  
4         println!("Data: {data}"); // `data` moved into closure, now owned by thread  
5     });  
6     handle.join().unwrap();  
7 }
```

([playground link](#))

Move can also be used for blocks (not closures):

1.9. Review: what is 'static'?

In the past sessions we have seen that `T: 'static` means:

Definition 1.9.1. *A type `T` is 'static if it contains no non-'static references.*

Things that are 'static:

- Owned data: `String`, `Vec<T>`, `Box<T>`, etc.
- 'static references: `&'static str`, `&'static T`

Things that are not 'static:

- References with shorter lifetimes: `&T`, `&'a T` where `'a` is not 'static
- Types with non-'static fields: `&T`, `&'a T`, `Vec<&T>`, etc.

1.9. Review: what is 'static?

In the past sessions we have seen that `T: 'static` means:

Definition 1.9.1. *A type `T` is 'static if it contains no non-'static references.*

Things that are 'static:

- Owned data: `String`, `Vec<T>`, `Box<T>`, etc.
- 'static references: `&'static str`, `&'static T`

Things that are not 'static:

- References with shorter lifetimes: `&T`, `&'a T` where `'a` is not 'static
- Types with non-'static fields: `&T`, `&'a T`, `Vec<&T>`, etc.

Strings are owned and without references, so they are 'static and can be moved

1.9. Review: what is 'static?

In the past sessions we have seen that T: 'static means:

Definition 1.9.1. *A type T is 'static if it contains no non-'static references.*

Things that are 'static:

- Owned data: String, Vec<T>, Box<T>, etc.
- 'static references: &'static str, &'static T

Things that are not 'static:

- References with shorter lifetimes: &T, &'a T where 'a is not 'static
- Types with non-'static fields: &T, &'a T, Vec<&T>, etc.

Strings are owned and without references, so they are 'static and can be moved

1.9.1. Spawn function

The standard library spawn function looks like this (simplified):

```
1 fn spawn<F, T>(f: F) -> JoinHandle<T>
2 where
3     F: FnOnce() -> T + 'static,
4     T: 'static
```

([playground link](#))

(Similar for asynchronous spawns like tokio::spawn, see next session.)

1. Threads	2
2. Channels	13
3. Channels	14
4. Send and Sync	19
5. Shared state	28
6. Exercises	35

1. Threads	2
2. Channels	13
3. Channels	14
3.1. What is a channel?	15
3.2. N-to-1 Channels	16
3.3. Bounded channels	17
3.4. Exercises	18
4. Send and Sync	19
5. Shared state	28
6. Exercises	35

3.1. What is a channel?

A channel is a communication primitive for passing messages between threads.

Under the hood: a thread-safe buffer (queue) in shared memory with:

- **Sender** (tx): pushes messages into the buffer
- **Receiver** (rx): pulls messages from the buffer



3.1. What is a channel?

A channel is a communication primitive for passing messages between threads.

Under the hood: a thread-safe buffer (queue) in shared memory with:

- **Sender** (tx): pushes messages into the buffer
- **Receiver** (rx): pulls messages from the buffer



Key properties:

- **Decouples** sender and receiver (no shared mutable state)
- **Asynchronous**: sender doesn't wait for receiver (unless bounded)
- **Ownership transfer**: messages are moved, not shared

3.2. N-to-1 Channels

```
1  fn main() {
2      let (tx, rx) = std::sync::mpsc::channel();
3      std::thread::spawn(move || {
4          let thread_id = std::thread::current().id();
5          for i in 0..10 {
6              tx.send(format!("Message {i}")).unwrap();
7              println!("{thread_id:?}: sent Message {i}");
8          }
9          println!("{thread_id:?}: done");
10     });
11     std::thread::sleep(std::time::Duration::from_millis(100));
12
13     for msg in rx.iter() {
14         println!("Main: got {msg}");
15     }
16 }
```

[\(playground link\)](#)

In an unbounded channel:

- `send()` never blocks (a kind of “asynchronous” behavior)
- returns a `Result::Err` when all receivers have been dropped

Definition 3.2.1. A channel is “closed” when all receivers have been dropped.

3.3. Bounded channels

If you don't want the channel buffer to grow indefinitely and cause memory overflow, you can use a bounded channel:

3.3. Bounded channels

If you don't want the channel buffer to grow indefinitely and cause memory overflow, you can use a bounded channel:

```
1  fn main() {
2      let (tx, rx) = mpsc::sync_channel(3);
3
4      thread::spawn(move || {
5          let thread_id = thread::current().id();
6          for i in 0..10 {
7              tx.send(format!("Message {i}")).unwrap();
8              println!("{thread_id:?}: sent Message {i}");
9          }
10         println!("{thread_id:?}: done");
11     });
12     thread::sleep(Duration::from_millis(100));
13
14     for msg in rx.iter() {
15         println!("Main: got {msg}");
16     }
17 }
```

(playground link)

This channel provides an additional method `try_send()` that returns immediately with an error if the buffer is full.

3.4. Exercises

- `examples/channel.rs`

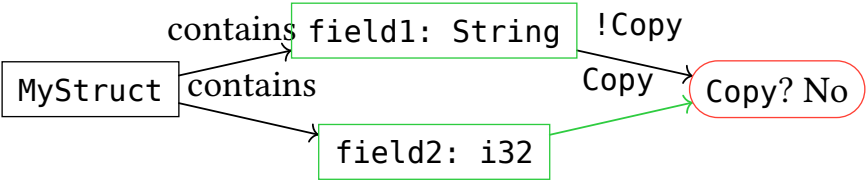
1. Threads	2
2. Channels	13
3. Channels	14
4. Send and Sync	19
4.1. Marker Traits	20
4.2. Traits for thread safety	21
4.3. Send	22
4.4. Sync	23
4.5. Examples	24
4.6. Summary	27
5. Shared state	28
6. Exercises	35

What are auto traits?

4.1. Marker Traits

What are auto traits?

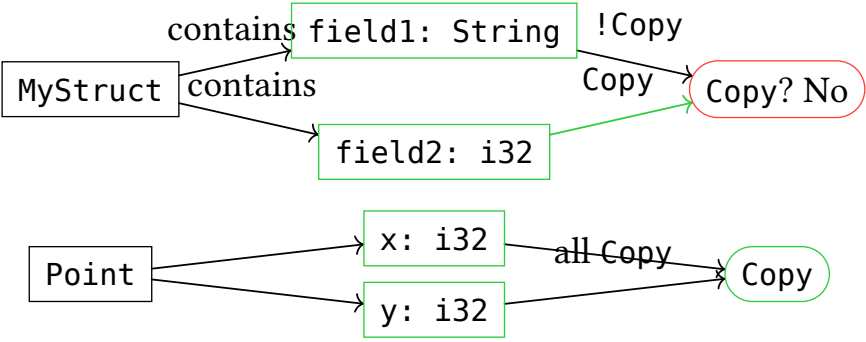
The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.



4.1. Marker Traits

What are auto traits?

The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.

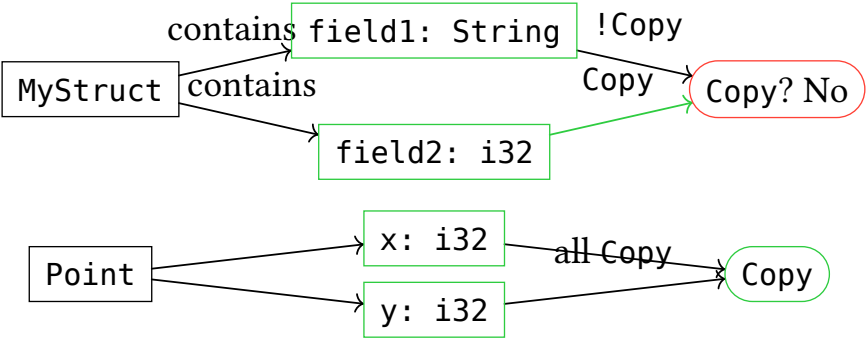


What are unsafe traits?

4.1. Marker Traits

What are auto traits?

The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.



What are unsafe traits?

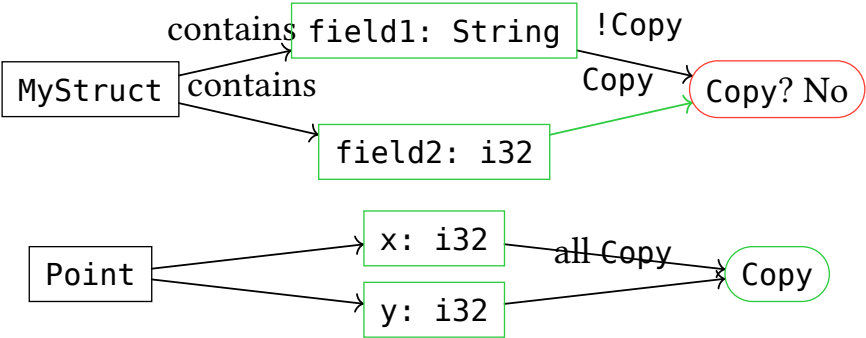
Traits that have safety invariants that the compiler cannot verify automatically. Implementing them incorrectly can lead to undefined behavior.

You can implement unsafe traits manually when you know it is valid.

4.1. Marker Traits

What are auto traits?

The compiler will automatically derive them for your types as long as they only contain types that implement the auto trait.



What are unsafe traits?

Traits that have safety invariants that the compiler cannot verify automatically. Implementing them incorrectly can lead to undefined behavior.

You can implement unsafe traits manually when you know it is valid.

Warning

Implementing unsafe traits requires unsafe blocks. Using them as constraints does not.

4.2. Traits for thread safety

How does Rust know to forbid shared access across threads?

4.2. Traits for thread safety

How does Rust know to forbid shared access across threads?

- Send: a type T is Send if it is safe to move a T across a thread boundary.

4.2. Traits for thread safety

How does Rust know to forbid shared access across threads?

- Send: a type T is Send if it is safe to move a T across a thread boundary.
- Sync: a type T is Sync if it is safe to move a &T across a thread boundary.

4.2. Traits for thread safety

How does Rust know to forbid shared access across threads?

- Send: a type T is Send if it is safe to move a T across a thread boundary.
- Sync: a type T is Sync if it is safe to move a &T across a thread boundary.

Info

Send and Sync are unsafe auto traits.

4.3. Send

A type `T` is `Send` if it is **safe to move a `T` value to another thread**.

4.3. Send

A type `T` is `Send` if it is **safe to move a `T` value to another thread**.

Moving ownership entails:

- running destructor in the other thread
- accessing the value in the other thread
- taking mutable references to the value in the other thread

4.3. Send

A type T is Send if it is **safe to move a T value to another thread**.

Moving ownership entails:

- running destructor in the other thread
- accessing the value in the other thread
- taking mutable references to the value in the other thread

```
1  use std::thread;
2
3  fn main() {
4      let data = vec![1, 2, 3]; // Vec<i32> is Send
5
6      let handle = thread::spawn(move || {
7          println!("Data in thread: {:?}", data);
8          data.iter().sum::<i32>()
9      });
10
11     let result = handle.join().unwrap();
12     println!("Sum: {}", result);
13 }
```

(playground link)

This compiles because Vec<i32> implements Send.

4.4. Sync

A value of type T is Sync if and only if an immutable reference to T ($\&T$) is Send.

In practice this means that it is safe to **access immutably from several threads in parallel**.

Corollary 4.4.0.1. *A consequence of this is that if $T: !\text{Sync}$ then $\&T: !\text{Send}$.*

4.4. Sync

A value of type T is Sync if and only if an immutable reference to T ($\&T$) is Send.

In practice this means that it is safe to **access immutably from several threads in parallel**.

Corollary 4.4.0.2. *A consequence of this is that if $T: !\text{Sync}$ then $\&T: !\text{Send}$.*

Proof.

$$\&T \Leftrightarrow \&\&T$$

$$T : \text{Sync} \Leftrightarrow \&T : \text{Send}$$



4.5. Examples

4.5.1. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability:** i32, bool, String, Vec<T>

4.5. Examples

4.5.1. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability:** `i32`, `bool`, `String`, `Vec<T>`
- **Thread-safe shared ownership:** `Arc<T>` (atomic ref counting)

4.5. Examples

4.5.1. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability:** `i32`, `bool`, `String`, `Vec<T>`
- **Thread-safe shared ownership:** `Arc<T>` (atomic ref counting)
- **Hardware-supported atomics:** `AtomicBool`, `AtomicU8` (compare-and-swap, fetch-and-add)

When are the generic types such as `Vec<T>` Send + Sync

4.5. Examples

4.5.1. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability:** `i32`, `bool`, `String`, `Vec<T>`
- **Thread-safe shared ownership:** `Arc<T>` (atomic ref counting)
- **Hardware-supported atomics:** `AtomicBool`, `AtomicU8` (compare-and-swap, fetch-and-add)

When are the generic types such as `Vec<T>` Send + Sync

When the type parameters `T` are Send + Sync.

4.5.2. Send + !Sync

These types can be moved to other threads, but cannot be shared via `&T`:

4.5. Examples

4.5.1. Send + Sync

Most types you come across are Send + Sync:

- **Owned data, no interior mutability:** `i32`, `bool`, `String`, `Vec<T>`
- **Thread-safe shared ownership:** `Arc<T>` (atomic ref counting)
- **Hardware-supported atomics:** `AtomicBool`, `AtomicU8` (compare-and-swap, fetch-and-add)

When are the generic types such as `Vec<T>` Send + Sync

When the type parameters `T` are Send + Sync.

4.5.2. Send + !Sync

These types can be moved to other threads, but cannot be shared via `&T`:

- **Non-thread-safe interior mutability:** `Cell<T>`, `RefCell<T>`
 - Allow mutation through `&T` without synchronization
 - Safe to own exclusively on one thread, unsafe to share references

4.5. Examples

4.5.3. !Send + Sync

These types are safe to access (via shared references) from multiple threads, but cannot be moved to another thread:

- **Lock guards:** `MutexGuard`, `RwLockReadGuard`, `RwLockWriteGuard`
 - Must be unlocked on the thread that acquired the lock (POSIX requirement)

Proposition 4.5.1. *`MutexGuard<T: Sync>`: !Send + Sync*

4.5. Examples

4.5.3. !Send + Sync

These types are safe to access (via shared references) from multiple threads, but cannot be moved to another thread:

- **Lock guards:** `MutexGuard`, `RwLockReadGuard`, `RwLockWriteGuard`
 - Must be unlocked on the thread that acquired the lock (POSIX requirement)

Proposition 4.5.1. *`MutexGuard<T: Sync>: !Send + Sync`*

Proof. **Part 1:** `MutexGuard<T: Sync>: !Send`

$$\begin{aligned} \text{drop}(\text{MutexGuard}) &\Rightarrow \text{Mutex}::\text{unlock}() \\ \text{Mutex}::\text{unlock}() &\Rightarrow \text{must run on locking thread} \\ \text{MutexGuard} : \text{Send} &\Rightarrow \text{drop can run on any thread} \\ \therefore \text{MutexGuard} &: !\text{Send} \end{aligned}$$

Part 2: `MutexGuard<T: Sync>: Sync`

$$\begin{aligned} \text{MutexGuard}<T> &: \text{Sync} \\ \Leftrightarrow \&\text{MutexGuard}<T> : \text{Send} \\ \Leftrightarrow \&T : \text{Send} & \text{ (deref coercion)} \\ \Leftrightarrow T &: \text{Sync} \quad \checkmark \end{aligned}$$

□

4.5. Examples

4.5.4. !Send + !Sync

These types are not thread-safe and cannot be moved to other threads:

4.5. Examples

4.5.4. !Send + !Sync

These types are not thread-safe and cannot be moved to other threads:

- dereferences and has non-atomic count: `Rc<T>`
- considered unsafe in general `*const T`, `*mut T`

4.6. Summary

	Sync	!Sync
Send	i32, bool, String, Vec<T> Arc<T>, Mutex<T>, Atomic* Safe to move and share	Cell<T>, RefCell<T> Interior mutability: safe to move, not share
!Send	MutexGuard, RwLock*Guard Thread-bound: safe to share, not move	Rc<T>, *const T, *mut T Not thread-safe at all

1. Threads	2
2. Channels	13
3. Channels	14
4. Send and Sync	19
5. Shared state	28
5.1. Arc	29
5.2. Mutex	31
5.3. Interior mutability	33
5.4. Example	34
6. Exercises	35

5.1. Arc

```
1  use std::sync::Arc; use std::thread;
2  #[derive(Debug)]
3  struct WhereDropped(Vec<i32>); /// A struct that prints which thread drops it.
4  impl Drop for WhereDropped {
5      fn drop(&mut self) { println!("Dropped by {:?}", thread::current().id()) }
6  }
7  fn main() {
8      let v = Arc::new(WhereDropped(vec![10, 20, 30]));
9      let mut handles = Vec::new();
10     for i in 0..5 {
11         let v = Arc::clone(&v);
12         handles.push(thread::spawn(move || {
13             // Sleep for 0-500ms.
14             std::thread::sleep(std::time::Duration::from_millis(500 - i * 100));
15             let thread_id = thread::current().id();
16             println!("{thread_id:?}: {v:?}");
17         }));
18     }
19     drop(v); // Now only the spawned threads will hold clones of `v`.
20     // When the last spawned thread finishes, it will drop `v`'s contents.
21     handles.into_iter().for_each(|h| h.join().unwrap());
22 }
```

[\(playground link\)](#)

What does `Arc<T>` stand for?

5.1. Arc

What does Arc<T> stand for?

Atomic Reference Counted.

Atomic refers to the fact that the **reference count is updated using atomic operations**, making atomics (AtomicUsize) and Arc<T> safe to share between threads.

When does Arc<T> implement Clone?

5.1. Arc

What does Arc<T> stand for?

Atomic Reference Counted.

Atomic refers to the fact that the **reference count is updated using atomic operations**, making atomics (AtomicUsize) and Arc<T> safe to share between threads.

When does Arc<T> implement Clone?

Always. Cloning an Arc only increments the reference count.

When does Arc<T> implement Send and Sync?

5.1. Arc

What does Arc<T> stand for?

Atomic Reference Counted.

Atomic refers to the fact that the **reference count is updated using atomic operations**, making atomics (AtomicUsize) and Arc<T> safe to share between threads.

When does Arc<T> implement Clone?

Always. Cloning an Arc only increments the reference count.

When does Arc<T> implement Send and Sync?

When T implements Send and Sync.

5.2. Mutex

Mutex allows mutable access to T behind a read-only interface:

Get `&mut T` from an `&Mutex<T>` by taking the lock.

```
1  use std::sync::Mutex;
2
3  fn main() {
4      let v = Mutex::new(vec![10, 20, 30]);
5      println!("v: {:?}", v.lock().unwrap());
6
7      {
8          let mut guard = v.lock().unwrap();
9          guard.push(40);
10     }
11
12     println!("v: {:?}", v.lock().unwrap());
13 }
```

(playground link)

5.2. Mutex

Mutex allows mutable access to T behind a read-only interface:

Get `&mut T` from an `&Mutex<T>` by taking the lock.

```
1  use std::sync::Mutex;
2
3  fn main() {
4      let v = Mutex::new(vec![10, 20, 30]);
5      println!("v: {:?}", v.lock().unwrap());
6
7      {
8          let mut guard = v.lock().unwrap();
9          guard.push(40);
10     }
11
12     println!("v: {:?}", v.lock().unwrap());
13 }
```

(playground link)

Exercise: `examples/arc-mutex.rs`

The `lock()` returns a `Result<MutexGuard<T>, PoisonError>`. What is a `PoisonError`?

The `lock()` returns a `Result<MutexGuard<T>, PoisonError>`. What is a `PoisonError`?

Another thread panicked while holding the lock

Why is `Mutex<T>: Sync` when `T: Send` (not `T: Sync`)?

5.2. Mutex

The `lock()` returns a `Result<MutexGuard<T>, PoisonError>`. What is a `PoisonError`?

Another thread panicked while holding the lock

Why is `Mutex<T>: Sync` when `T: Send` (not `T: Sync`)?

The mutex ensures exclusive access - only one thread touches `T` at a time, so `T` doesn't need to support concurrent sharing.

```
1 unsafe impl<T: ?Sized + Send> Sync for Mutex<T> {}
```

[\(playground link\)](#)

5.3. Interior mutability

Definition 5.3.1 (Interior mutability). *A type that allows mutation through a shared reference ($\&T \rightarrow \&\text{mut inner}$).*

5.3. Interior mutability

Definition 5.3.1 (Interior mutability). *A type that allows mutation through a shared reference ($\&T \rightarrow \&mut\ inner$).*

Types with interior mutability:

Type	Thread-safe?	Checking	Use case
<code>Cell<T></code>	No	Compile-time	Simple values, Copy types
<code>RefCell<T></code>	No	Runtime (panics)	Complex borrows, single-threaded
<code>Mutex<T></code>	Yes	Runtime (blocks)	Exclusive access across threads
<code>RwLock<T></code>	Yes	Runtime (blocks)	Many readers, few writers
<code>Atomic*</code>	Yes	Hardware	Counters, flags, lock-free

5.3. Interior mutability

Definition 5.3.1 (Interior mutability). *A type that allows mutation through a shared reference ($\&T \rightarrow \&mut\ inner$).*

Types with interior mutability:

Type	Thread-safe?	Checking	Use case
<code>Cell<T></code>	No	Compile-time	Simple values, Copy types
<code>RefCell<T></code>	No	Runtime (panics)	Complex borrows, single-threaded
<code>Mutex<T></code>	Yes	Runtime (blocks)	Exclusive access across threads
<code>RwLock<T></code>	Yes	Runtime (blocks)	Many readers, few writers
<code>Atomic*</code>	Yes	Hardware	Counters, flags, lock-free

5.4. Example

Typically Rust beginners will:

1. Wrap `T` in `Mutex::new(Arc::new(T))`
2. Clone the `Arc` to share ownership across threads
3. Lock the `Mutex` to get mutable access to `T` in each thread

There are some problems with this approach:

5.4. Example

Typically Rust beginners will:

1. Wrap `T` in `Mutex::new(Arc::new(T))`
2. Clone the `Arc` to share ownership across threads
3. Lock the `Mutex` to get mutable access to `T` in each thread

There are some problems with this approach:

- **Deadlocks**: the guard returned by `.lock()` must be dropped and a costly operation (like I/O) performed while holding the lock.

5.4. Example

Typically Rust beginners will:

1. Wrap `T` in `Mutex::new(Arc::new(T))`
2. Clone the `Arc` to share ownership across threads
3. Lock the `Mutex` to get mutable access to `T` in each thread

There are some problems with this approach:

- **Deadlocks**: the guard returned by `.lock()` must be dropped and a costly operation (like I/O) performed while holding the lock.
- **Coarse locking**: the whole data structure is locked, even when only a small part needs to be mutated.

5.4. Example

Typically Rust beginners will:

1. Wrap `T` in `Mutex::new(Arc::new(T))`
2. Clone the `Arc` to share ownership across threads
3. Lock the `Mutex` to get mutable access to `T` in each thread

There are some problems with this approach:

- **Deadlocks**: the guard returned by `.lock()` must be dropped and a costly operation (like I/O) performed while holding the lock.
- **Coarse locking**: the whole data structure is locked, even when only a small part needs to be mutated.
- **Global mutability**: all threads can mutate the data, making reasoning about the program harder.

5.4. Example

Typically Rust beginners will:

1. Wrap `T` in `Mutex::new(Arc::new(T))`
2. Clone the `Arc` to share ownership across threads
3. Lock the `Mutex` to get mutable access to `T` in each thread

There are some problems with this approach:

- **Deadlocks**: the guard returned by `.lock()` must be dropped and a costly operation (like I/O) performed while holding the lock.
- **Coarse locking**: the whole data structure is locked, even when only a small part needs to be mutated.
- **Global mutability**: all threads can mutate the data, making reasoning about the program harder.

My advice:

- Minimise or remove usage of `Arc<Mutex<T>>`.
- Use `RwLock<T>` when possible.
- Use `Atomic` types for simple shared counters/flags.

1. Threads	2
2. Channels	13
3. Channels	14
4. Send and Sync	19
5. Shared state	28
6. Exercises	35

From Google:

- [examples/philosophers.rs](#)
- [examples/link.rs](#)